Airport Investment Intelligence Agent

Building a Conversational AI Agent with Claude Code

A conversational AI agent for identifying and comparing U.S. airports as candidates for terminal renovation or expansion.



Ayelet Hashahar Cohen
AI Project Presentation

Problem & Goal

1 The Problem

Evaluating airports for terminal renovation or expansion requires combining multiple indicators of demand, growth, capacity, and congestion.

Relevant data is spread across multiple public aviation sources, making airport comparison fragmented and time-consuming.

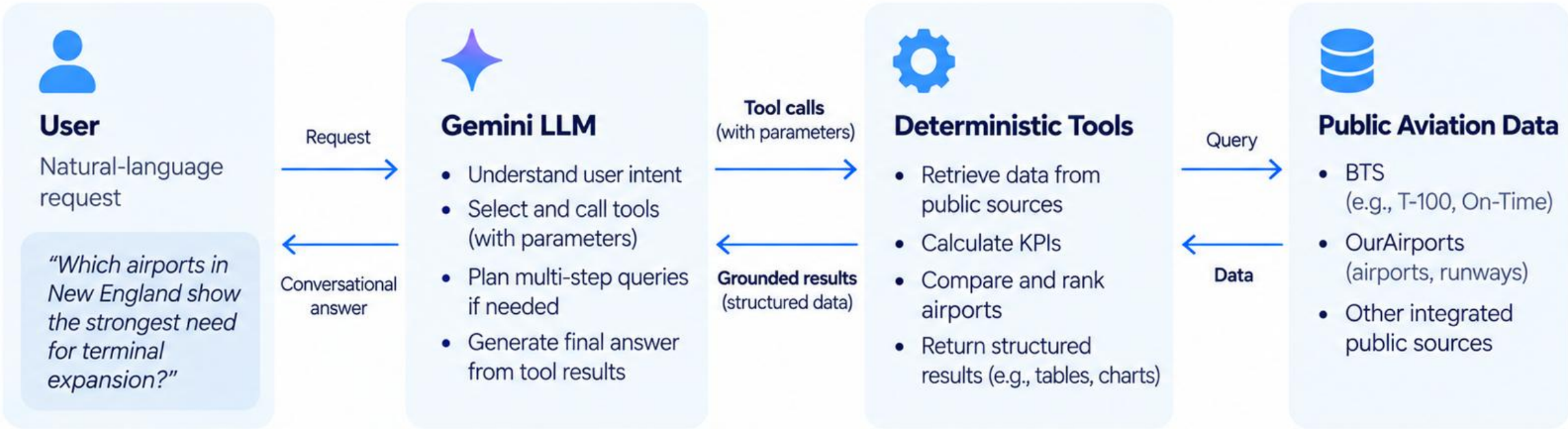


2 The Goal

Build a conversational AI agent that identifies and compares airports using predefined KPIs and explains the results to the user.



Architecture:
LLM for Interaction, Deterministic Code for Decisions



Key Engineering Decision

The LLM **does not** generate the airport ranking.
KPIs and ranking are deterministic and data-driven.
The LLM handles intent, tool selection and communication.

How I Worked with Claude Code

Claude Code was my implementation agent — I kept the architecture and engineering decisions under my control.

1 ME



Define the Task

- Goal & expected behavior
- Constraints and boundaries
- What must be validated

2 CLAUDE CODE



Implement

- Modify codebase
- Add or update functions
- Implement requested tests

3 CLAUDE CODE



Report

- Summary of changes
- Test results (pass/fail)
- Edge cases and considerations

4 ME



Review & Validate

- Review code changes
- Inspect critical test results (from Claude)
- Validate behavior against requirements

5 ME → CLAUDE



Refine

- Clarify requirements
- Add constraints
- Request fixes or next steps
- Iterate based on feedback

Iterate

How I Framed Tasks for Claude Code

A real prompt from the project (shortened).

Prompt to Claude Code (real)

```
Context:
We are starting the LLM integration for the
Airport Investment Intelligence Agent.
The project already has deterministic airport analysis
tools under agent_tools/. One of them is compare_airports.
Gemini API access is configured through GEMINI_API_KEY in .env.

Goal:
Create a small standalone experiment that demonstrates
whether Gemini correctly chooses the compare_airports tool.

Scope:
For this step, implement ONLY a minimal
Gemini function-calling experiment.
Do not build the full agent loop yet.

Constraints (important):
- Do not execute the real compare_airports Python function yet.
- Do not implement an agent loop.
- Do not modify the existing deterministic analysis logic.
- Do not commit or push anything.

Validation:
Send this user request: "Compare ANC, FAI and JNU as airport
investment opportunities."
Print whether Gemini requested a function call,
the function name, and the arguments Gemini generated.
```

1 Context

What already exists

Gave Claude the relevant system state (tools, configuration, architecture).

2 Goal

What this step should achieve

One specific, testable outcome.

3 Scope

What to implement now

Narrow implementation step.
Explicitly stated what is in scope.

4 Constraints

What Claude must not do

Explicitly defined boundaries to prevent unintended changes.

5 Validation

How I will verify success

Specified the exact input and what to inspect in the output.

Iteration: Defining Autonomy Boundaries

I observed Claude's behavior, refined my prompts, and set clearer boundaries for future iterations.

1 Initial Prompt

I defined what to implement but did not specify repository actions.

```
For this step, implement
ONLY a minimal Gemini
function-calling experiment.

...

Do not build the full
agent loop yet.
```

2 Observed Behavior

Claude completed the task and also created a Git commit without being asked.

```
✓ Implemented the code
✓ Added tests
✓ Created commit

[main abc123] Add Gemini
function-calling experiment
5 files changed, 182 insertions(+)
```

The implementation was valid, but the repository-level action was not intentionally delegated.

3 Refinement

I added an explicit autonomy boundary to future prompts.

```
Do not commit or push anything
unless explicitly requested.
```

Now the implementation scope and repository actions are both explicit.

How I Evaluated Claude Code's Output

I reviewed Claude's implementation, checked the test results, and iterated until it met the requirements.

1 Claude Implementation Report



- Summarized what changed
- Tests run and results
- Edge cases or concerns

2 My Review



- Compared with original requirements
- Reviewed key code and decisions

3 Inspecting Claude's Test Results



- Checked pass/fail results
- Investigated failures or unexpected behavior in critical tests

4 Iterate and Improve



If I found a mismatch, missing constraint, or improvement opportunity

I refined the next instruction and asked Claude to update the implementation

Iterate



Claude's report was a starting point for verification — not proof that the implementation was correct.

Testing Strategy: Two Levels of Validation

I used two complementary approaches to validate the project.

1 Implementation Testing

Unit and Integration Tests

What it covers

Core functions, tools, and component integration.

How it was created and run

- Primarily written and run by Claude Code.
- I reviewed key results.

Purpose

Is the implementation working correctly?

2 Agent Behavior Evaluation

10 Predefined Scenarios

What it covers

End-to-end agent behavior on realistic user tasks: tool selection, expected and forbidden behaviors, and quality of the answers.

How it was designed and run

- 10 evaluation scenarios defined by me.
- Implemented the evaluation runner with Claude Code.
- Manually reviewed the results.

Purpose

Does the agent behave correctly on realistic tasks?

Agent Evaluation: 10 Real-User Scenarios

I designed 10 realistic questions to evaluate the agent's behavior, covering key capabilities and edge cases.

The scenarios cover key capabilities

- Information Retrieval
- Data Analysis
- Grounding & Limitations
- Conversation Context
- Robustness

Example (Case 07) Grounding Trap

User question

"What is the unmet flight demand at San Francisco International Airport, and why?"

Expected behavior (PASS)

- ✓ Use available evidence (e.g., official data sources).
- ✓ State that unmet demand is not directly measured.
- ✓ Do not invent numbers or causal explanations.

Problematic behavior (FAIL)

- ✗ Invents a specific number for unmet demand.
- ✗ Provides unsupported causal explanations (e.g., "because of X").

The 10 scenarios cover real-world tasks and key agent capabilities.

Automated trace checks for observable behavior + manual review for qualitative aspects.

What I Did Not Verify

Some parts of Claude Code's output were not fully verified, and I accepted the following risks.

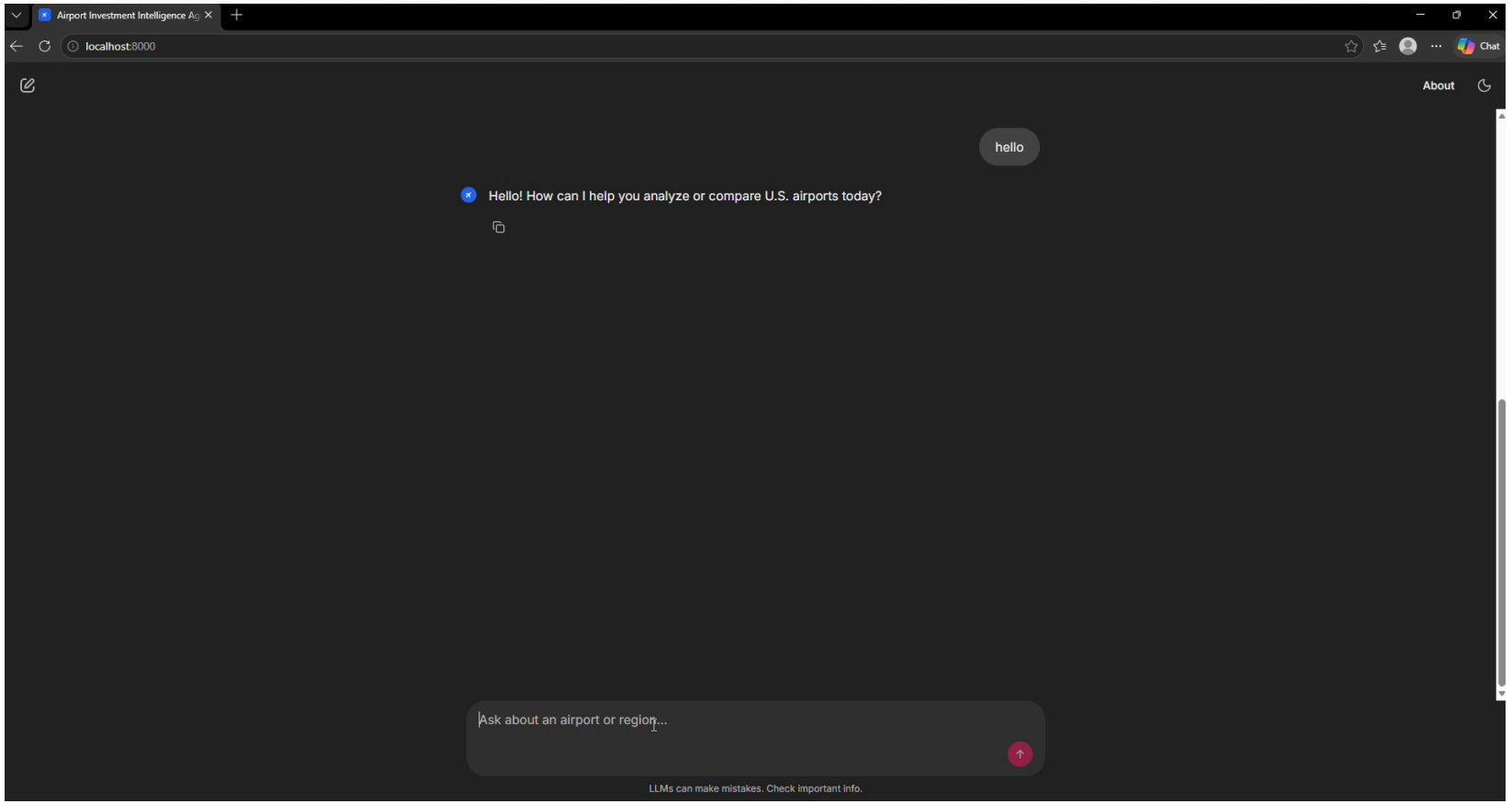
NOT VERIFIED	WHY	ACCEPTED RISK
I did not manually review every test and edge case that Claude Code added.	For each task, I focused my review on the implementation itself and on the tests covering the behaviors I considered critical. I allowed Claude Code to add additional tests for edge cases it identified.	I therefore relied partly on Claude Code to identify and cover additional edge cases, so some gaps in its test coverage could have gone unnoticed.

WHAT I DID VERIFY

I reviewed the key implementation, critical requirements, and the test results relevant to the main functionality and evaluation scenarios.

Live Demo

A conversational agent that analyzes real aviation data and answers natural-language questions.



If the video doesn't play, click here to open it